

대화 히스토리 압축 및 컨텍스트 관리



1. 개요

멀티턴(Multi-turn) AI Agent와 RAG 시스템은 사용자의 이전 대화를 기억하여 연속적인 대화를 수행한다.

매 요청마다 LLM으로 전달되는 프롬프트는 일반적으로 다음과 같이 구성된다.

```
System Prompt
+
Conversation History
+
Retrieved Documents(RAG)
+
Current User Question
```



대화가 길어질수록 Conversation History가 계속 증가하면서 다음과 같은 문제가 발생한다.

- Context Window 부족
- 입력 토큰 증가
- API 비용 증가
- 응답 속도 저하
- Lost in the Middle 현상 발생

따라서 AI Agent는 과거 대화를 적절히 압축하고 필요한 정보만 유지하는 Context Management 전략이 필요하다.

2. Context Window와 토큰 예산(Token Budget)

LLM은 한 번에 처리할 수 있는 최대 입력 길이(Context Window)를 가진다.

예를 들어 Context Window가 128K Token인 모델에서

System Prompt	5K
History	60K
RAG Docs	50K
Question	2K

Total	117K

이라면 정상적으로 처리된다.

하지만

System Prompt	5K
History	90K
RAG Docs	50K
Question	2K

Total	147K

가 되면 Context Window를 초과하게 된다.

따라서 실제 AI Agent는 항상 Token Budget을 계산하여 입력을 구성한다.

3. 왜 토큰을 기준으로 관리해야 하는가?

메시지 개수와 토큰 수는 비례하지 않는다.

예를 들어

안녕하세요.

는 몇 개의 토큰에 불과하지만

500줄짜리 Python 코드

는 수천 개의 토큰이 될 수 있다.

따라서

- 메시지 개수
- 문자 수

보다

- Token 수

를 기준으로 관리하는 것이 더욱 정확하다.

4. 대표적인 히스토리 압축 전략

AI Agent에서 가장 많이 사용하는 전략은 다음과 같다.

전략	핵심 아이디어	장점	단점
Sliding Window	최근 N개 대화 유지	단순하고 빠름	오래된 정보 삭제
Token Budget Trim	토큰 상한까지 유지	Context Window 보호	중요한 정보 삭제 가능
Summarization	오래된 대화 요약	장기 정보 유지	요약 품질 영향
Hybrid	요약 + 최근 원문	실무에서 가장 많이 사용	구현 복잡
Running Summary	기존 요약을 지속 갱신	비용 절감	요약 관리 필요

5. Sliding Window

가장 단순한 방법이다.

최근 N개의 대화만 유지한다.

대화1

대화2

대화3

대화4

대화5

대화6

Window = 3



대화4

대화5

대화6

장점

- 구현이 매우 간단
- 추가 LLM 호출 없음
- 빠른 응답

단점

- 초반의 중요한 정보가 삭제될 수 있다.

예)

- 사용자 이름
- 프로젝트 정보
- 선호 설정

6. Token Budget Trim

메시지 개수가 아니라 토큰 예산을 기준으로 대화를 유지한다.

예)

최대 8,000 Token

현재

7,700 Token

새로운 질문

600 Token

↓

8,300 Token

↓

↓

오래된 대화를 삭제하여

7,900 Token

으로 맞춘다.

LangChain에서는 `trim_messages()`를 이용하여 쉽게 구현할 수 있다.

장점

- Context Window를 넘지 않음
- 토큰 기준으로 자동 관리

단점

- 오래된 중요한 정보가 삭제될 수 있음

7. Summarization

삭제 대신 정보를 압축한다.

예를 들어

원본

사용자 이름은 홍길동

Python 사용

RAG 프로젝트 진행

Qdrant 사용

답변은 한국어



요약

홍길동은 Python 기반 RAG 프로젝트를 수행하며 Qdrant를 사용하고 한국어 답변을 원한다.

LLM에는

요약

+

최근 대화

만 전달한다.

장점

- 장기 정보 유지
- 토큰 절감

단점

- 요약 비용 발생
- 요약 품질에 따라 정보 손실 가능

실무에서는 요약 프롬프트에 다음 내용을 반드시 포함하도록 한다.

- 고유명사
- 숫자
- 확정된 결정사항
- 제약조건

8. Hybrid Memory

실무에서 가장 많이 사용하는 방식이다.

최근 대화

+

요약본

+

현재 질문

필요하면 여기에

Vector Search 결과

도 함께 추가한다.

Summary

+

Recent Messages

+

Retrieved Memory

+

Current Question

장점

- 최근 대화 유지
- 장기 기억 유지
- 토큰 절약

9. Running Summary

매 요청마다 전체 대화를 다시 요약하면 비용이 계속 증가한다.

Running Summary는

기존 요약

+

새로운 대화

만 다시 요약한다.

Summary₁

+

New Messages

↓

Summary₂

이 방식을 반복하면

- 비용 감소
- 응답 속도 향상
- 긴 세션 지원

이 가능하다.

10. RAG에서의 Context Budget 관리

대화형 RAG에서는 히스토리뿐 아니라 검색 문서도 함께 관리해야 한다.

일반적으로 다음과 같이 토큰 예산을 분배한다.

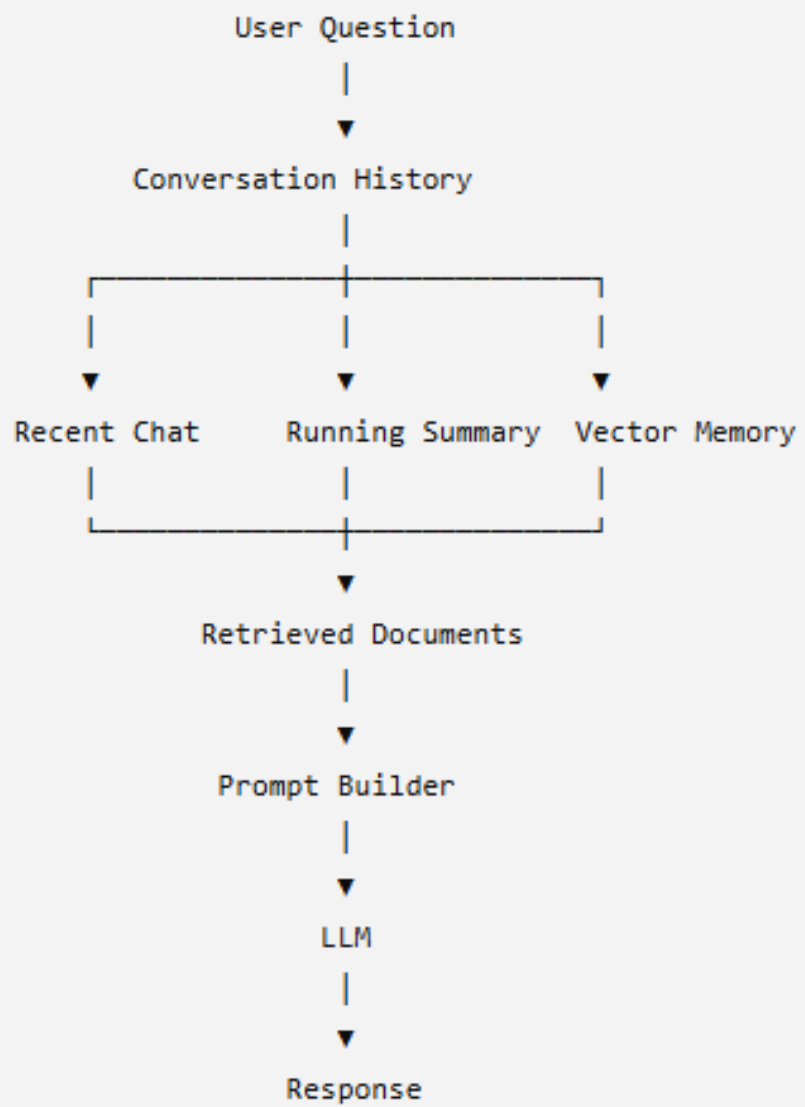
구성 요소	권장 비율
System Prompt	5~10%
대화 히스토리	20~30%
RAG 문서	40~50%
현재 질문 및 출력 예약	15~20%

검색 문서가 너무 많을 경우에는

- Top-K 감소
- Reranking
- Contextual Compression
- 문서 요약

등을 함께 적용한다.

11. 전체 아키텍처



12. 실무 체크리스트

- 저장(Storage)과 프롬프트 입력(Context)은 분리하여 관리한다.
- 메시지 개수보다 토큰 수(Token Budget)를 기준으로 압축을 수행한다.
- 요약 시에는 고유명사, 숫자, 제약조건, 결정사항을 우선 보존한다.
- 최근 대화는 원문으로 유지하고 오래된 대화는 요약하는 Hybrid 방식을 우선 고려한다.
- 장기 기억이 필요한 경우에는 Vector Memory와 RAG를 함께 사용한다.
- 대화 히스토리 예산과 검색 문서 예산을 별도로 관리하여 Context Window를 효율적으로 활용한다.



감사합니다